

Projeto e Análise de Algoritmos I

Aula 05 - Notação Assintótica

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado, André Grahl Pereira, Lucas Nunes Alegre, Marcus Ritt e Luciana Buriol.

Análise Assintótica

Intuição

Ideia: Suprimir fatores constantes e termos de baixa ordem.

- Independente de máquina.
- Foco em grandes entradas.
- Descreve o crescimento de funções.
- Descreve o comportamento de funções **no limite**.

Exemplo: $8n^3 + 50n + 86$ cresce como n^3 .

Terminologia: O algoritmo tem tempo de execução $O(n^3)$, onde n é o tamanho da entrada.

Intuição

Problema: O vetor A contém o número t ?

BUSCA(A, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  return false
```

P: Qual é o tempo de execução?

P: Quanto espaço é usado?

Intuição

Problema: O vetor A contém o número t ?

BUSCA(A, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  return false
```

P: Qual é o tempo de execução? $O(n)$

Intuição

Problema: O vetor A ou o vetor B contém o número t ?

BUSCA-DUPLA(A, B, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  for  $i \leftarrow 1$  to  $n$  do
5:      if  $B[i] = t$  then
6:          return true
7:  return false
```

P: Qual é o tempo de execução?

Intuição

Problema: O vetor A ou o vetor B contém o número t ?

BUSCA-DUPLA(A, B, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  for  $i \leftarrow 1$  to  $n$  do
5:      if  $B[i] = t$  then
6:          return true
7:  return false
```

P: Qual é o tempo de execução? $O(n)$

Intuição

Problema: O vetor A e o vetor B contêm um número em comum?

BUSCA-COMUM(A, B, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow 1$  to  $n$  do
3:          if  $A[i] = B[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução?

Intuição

Problema: O vetor A e o vetor B contêm um número em comum?

BUSCA-COMUM(A, B, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow 1$  to  $n$  do
3:          if  $A[i] = B[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução? $O(n^2)$

Intuição

Problema: O vetor A tem um elemento duplicado?

VERIFICA-DUPLICATAS(A, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow i + 1$  to  $n$  do
3:          if  $A[i] = A[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução?

Intuição

Problema: O vetor A tem um elemento duplicado?

VERIFICA-DUPLICATAS(A, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow i + 1$  to  $n$  do
3:          if  $A[i] = A[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução? $O(n^2)$

Análise de Algoritmos

“Análise de algoritmos usualmente significa: atribuir uma expressão Big O para o tempo de execução. (Claro, Big Θ seria muito melhor.)”

— Ian Parberry. “*Problems on Algorithms*”, 1994, p. 59.

Notação Big *O* - Limitante Superior

Notação Big O - Limitante Superior

Obs. Seja $T(n)$ uma função em $n = 1, 2, 3, \dots$ (em geral o pior tempo de execução de um algoritmo).

P: Quando $T(n)$ é $O(f(n))$?

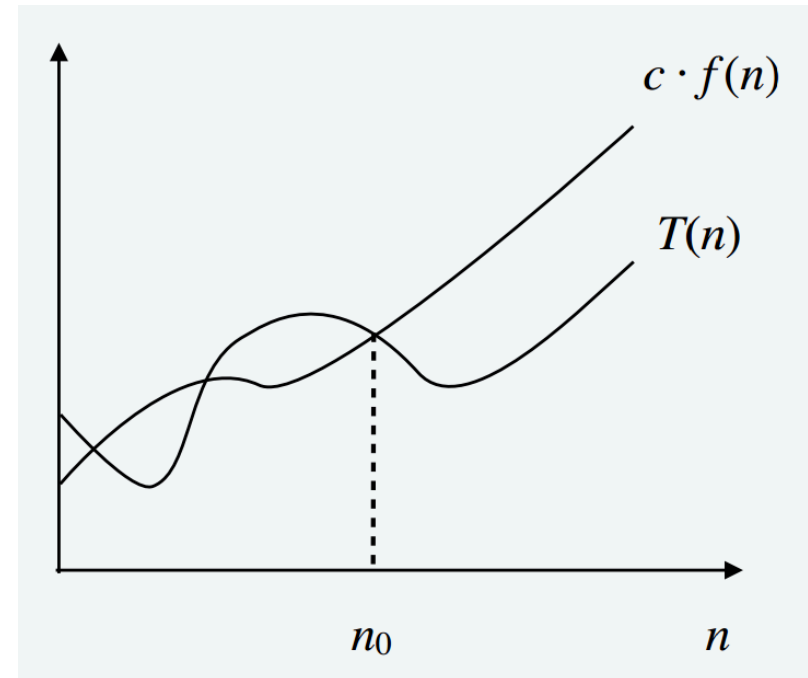
R: Se eventualmente (para n suficientemente grande), $T(n)$ é limitado por cima por $f(n)$ multiplicada por uma constante.

Notação Big O - Limitante Superior

Def. $T(n)$ é $O(f(n))$ se e somente se **existe** uma constante $c > 0$ e um $n_0 \geq 0$ tal que $T(n) \leq c \cdot f(n)$ **para todo** $n \geq n_0$.

Exemplo: $T(n) = 32n^2 + 17n + 1$

- $T(n)$ é $O(n^2)$.
- $T(n)$ é $O(n^3)$.
- $T(n)$ **não** é $O(n)$.
- $T(n)$ **não** é $O(n \log(n))$.
- **Uso:** Ordenação por Inserção é $O(n^2)$.



Obs. c e n_0 não podem depender de n .

Busca em Vetor Não Ordenado

BUSCA-LINEAR(x, A, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

$$T(n) = 3n + 1$$

$T(n)$ é $O(n)$: $c = 4$ e $n_0 = 1$, $c = 3,5$ e $n_0 = 2$, ...

$T(n)$ é $O(n^2)$, $O(n^3)$, ...

<https://www.geogebra.org/classic/czf4djzt>

BubbleSort

BUBBLESORT(A)

```
1:   $n \leftarrow \text{length}(A)$ 
2:  for  $i \leftarrow 1$  to  $n - 1$  do
3:    for  $j \leftarrow 1$  to  $n - i$  do
4:      if  $A[j] > A[j + 1]$  then
        TROCA( $A, j, j + 1$ )
```

$$T(n) = (n - 1)(n - 1)(1 + 5) = 6n^2 - 12n + 6$$

$T(n)$ é $O(n^2)$: $c = 6$ e $n_0 = 0, \dots$

$T(n)$ é $O(n^3)$, $O(n^4)$, $\dots$

$T(n)$ não é $O(n)$.

<https://www.geogebra.org/classic/h2f2scrd>

Notação Big O : Polinômios

Def. $T(n)$ é $O(f(n))$ se e somente se **existe** uma constante $c > 0$ e um n_0 tal que $T(n) \leq c \cdot f(n)$ **para todo** $n \geq n_0$.

Afirmção. Se $T(n) = a_k n^k + \dots + a_1 n^1 + a_0$ então $T(n) = O(n^k)$.

Prova. Escolha $n_0 = 1$ e $c = |a_k| + \dots + |a_1| + |a_0|$.

Nós temos que mostrar que $\forall n \geq n_0, T(n) \leq c \cdot n^k$.

$$T(n) \leq |a_k| n^k + \dots + |a_1| n^1 + |a_0|.$$

$$T(n) \leq |a_k| n^k + \dots + |a_1| n^k + |a_0| n^k.$$

$$T(n) \leq (|a_k| + \dots + |a_1| + |a_0|) n^k.$$

$$T(n) \leq c \cdot n^k \blacksquare$$

<https://www.geogebra.org/classic/dhycydkx>

Notação Big O : Polinômios

Def. $T(n)$ é $O(f(n))$ se e somente se **existe** uma constante $c > 0$ e um n_0 tal que $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$.

Afirmção. Para todo $k \geq 1$, n^k não é $O(n^{k-1})$.

Prova. Prova por contradição. Suponha que $n^k = O(n^{k-1})$.

Então **existem** c, n_0 tal que $n^k \leq c \cdot n^{k-1}, \forall n \geq n_0$.

Mas dividindo ambos os lados por n^{k-1} :

$$\frac{n^k}{n^{k-1}} \leq \frac{c \cdot n^{k-1}}{n^{k-1}}, \forall n \geq n_0$$

$$n \leq c, \forall n \geq n_0.$$

O que é uma contradição (pois c é uma constante). ■

Notação Big O

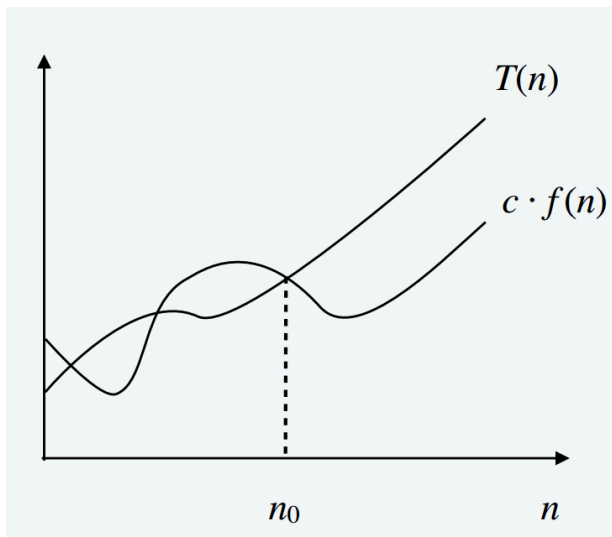
Sinal de Igual: $O(f(n))$ é um **conjunto de funções**, mas comumente escrevemos $T(n) = O(f(n))$ em vez de $T(n) \in O(f(n))$.

- O **domínio** de $f(n)$ em geral são os **números naturais**.
- OK abusar, mas não usar incorretamente.

Notação Big Ω (Omega) - Limitante Inferior

Notação Big Ω (Omega) - Limitante Inferior

Def. $T(n)$ é $\Omega(f(n))$ se e somente se **existe** uma constante $c > 0$ e um $n_0 \geq 0$ tal que $T(n) \geq c \cdot f(n)$ para todo $n \geq n_0$.



Exemplo: $T(n) = 32n^2 + 17n + 1$.

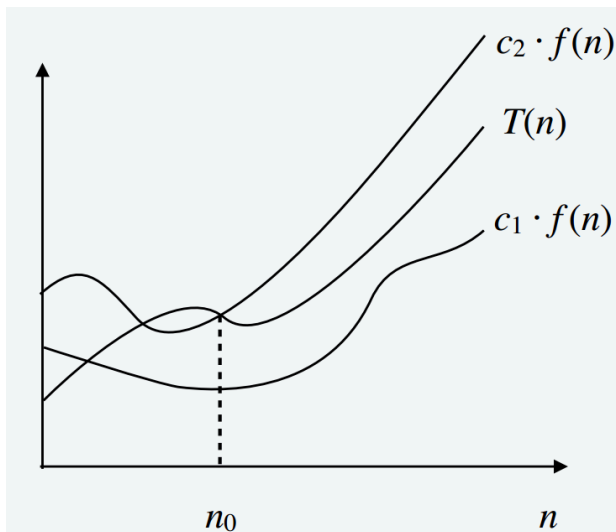
- $T(n)$ é $\Omega(n^2)$.
- $T(n)$ é $\Omega(n)$.
- $T(n)$ **não** é $\Omega(n^3)$.
- $T(n)$ **não** é $\Omega(n^3 \log(n))$.

Uso: Ordenação por Inserção é $\Omega(n)$.

Notação Big Θ (Theta) - Limites Precisos

Notação Big Θ (Theta) - Limites Precisos

Def. $T(n)$ é $\Theta(f(n))$ se e somente se **existem** constantes $c_1, c_2 > 0$ e um $n_0 \geq 0$ tal que $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ para todo $n \geq n_0$.



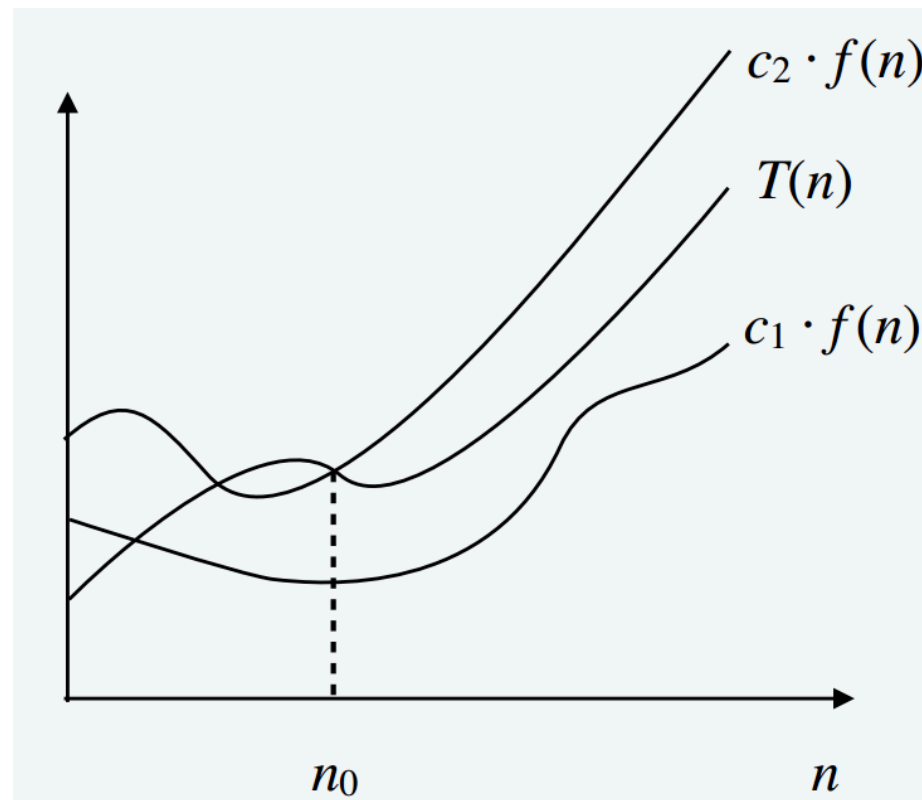
Exemplo: $T(n) = 32n^2 + 17n + 1$.

- $T(n)$ é $O(n^2)$.
- $T(n)$ é $\Omega(n^2)$.
- $T(n)$ é $\Theta(n^2)$.
- $T(n)$ **não** é $\Theta(n^3)$.
- $T(n)$ **não** é $\Theta(n)$.

Uso: BubbleSort é $\Theta(n^2)$.

Notação Big Θ (Theta) - Limites Precisos

Def. $T(n)$ é $\Theta(f(n))$ se e somente se $T(n) = O(f(n))$ e $T(n) = \Omega(f(n))$.



Análise Assintótica

Exemplo: Seja $T(n) = \frac{1}{2}n^2 + 3n$. Quais dos seguintes é verdadeiro (apresente constantes e n_0)?

- $T(n) = \mathcal{O}(n)$?
- $T(n) = \Omega(n)$?
- $T(n) = \mathcal{O}(n^3)$?
- $T(n) = \Theta(n^2)$?

Visualização Interativa: O , Ω e Θ

Demonstração Interativa: Comparação de Big O, Omega e Theta

Notação / Limitante:

Big-Theta [$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$]

Exemplos Pré-definidos:

$T(n) = 0.5n^2 + 3n$ | $f(n) = n^2$ (Exem

Função $T(n)$:

$0.5 \cdot n^2 + 3 \cdot n$

Função $f(n)$:

n^2

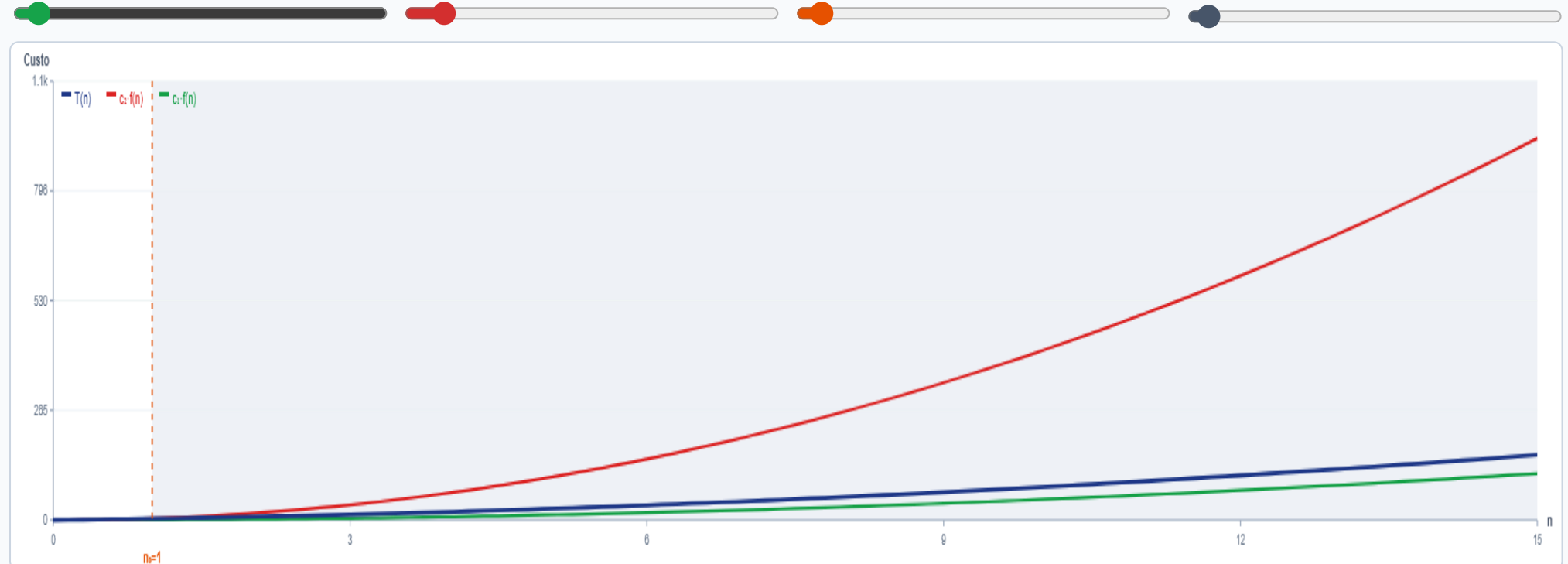
Fórmula Renderizada: $T(n) = 0.5n^2 + 3n$ | $f(n) = n^2$

Constante $c_1 = 0.5$

Constante $c_2 = 4.1$

Limiar $n_0 = 1$

Eixo $N_{\max} = 15$



✓ Válido: $0.5 \cdot f(n) \leq T(n) \leq 4.1 \cdot f(n) \quad \forall n \geq 1$

Análise Assintótica

Exemplo: Seja $T(n) = \frac{1}{2}n^2 + 3n$. Quais dos seguintes é verdadeiro (apresente constantes e n_0)?

- $T(n)$ não é $O(n)$.
- $T(n) = \Omega(n)$, $n_0 = 1$, $c = \frac{1}{2}$.
- $T(n) = O(n^3)$, $n_0 = 1$, $c = 4$.
- $T(n) = \Theta(n^2)$, $n_0 = 1$, $c_1 = \frac{1}{2}$, $c_2 = 4$.

Limites Estritos

Limites Estritos

- Além das notações O , Ω e Θ , existem duas notações para representar **limites estritos**.
- Essas duas notações, todavia, são raramente encontradas na literatura de computação.
- Nos próximos slides, apresentamos a notação o e ω para completude, porém, **não as utilizaremos nesta disciplina**.

Notação Pequeno o - Limitante Superior Estrito

Def. $T(n)$ é $o(f(n))$ se e somente se **para toda** constante $c > 0$ **existe** uma constante $n_0 > 0$ tal que $T(n) < c \cdot f(n)$ **para todo** $n \geq n_0$.

Exemplo: $T(n) = 32n^2 + 17n + 1$.

- $T(n)$ é $o(n^3)$.
- $T(n)$ **não** é $o(n^2)$.

Uso: BubbleSort é $o(n^3)$, mas **não** é $o(n^2)$.

Notação Pequeno ω (Omega) - Limitante Inferior Estrito

Def. $T(n)$ é $\omega(f(n))$ se e somente se **para toda** constante $c > 0$ **existe** uma constante $n_0 > 0$ tal que $T(n) > c \cdot f(n)$ **para todo** $n \geq n_0$.

Exemplo: $T(n) = 32n^2 + 17n + 1$.

- $T(n)$ é $\omega(n)$.
- $T(n)$ **não** é $\omega(n^2)$.

Uso: BubbleSort é $\omega(n)$.

Resumo das Notações Assintóticas

Resumo das Notações Assintóticas

- $f(n) = \mathbf{O}(g(n)) \approx f(n) \leq g(n)$
 - f cresce **no máximo tão rápido** quanto g , é um limitante superior.
- $f(n) = \mathbf{\Omega}(g(n)) \approx f(n) \geq g(n)$
 - f cresce **no mínimo tão rápido** quanto g , é um limitante inferior.
- $f(n) = \mathbf{\Theta}(g(n)) \approx f(n) = g(n)$
 - f cresce **tão rápido** quanto g , é um limitante preciso.
- $f(n) = o(g(n)) \approx f(n) < g(n)$
 - f cresce **mais devagar** que g , é um limitante superior estrito.
- $f(n) = \omega(g(n)) \approx f(n) > g(n)$
 - f cresce **mais rápido** que g , é um limitante inferior estrito.

Definições Alternativas (Limites)

Definições Alternativas (Limites)

$$f(n) = \mathbf{O}(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$$

$$f(n) = \mathbf{\Omega}(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$$

$$f(n) = \mathbf{\Theta}(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0$$

$$f(n) = o(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

$$f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

História

Quem Culpar?

“Well, I think I have beat this issue to death, knowing of no other arguments pro or con the introduction of Θ and Ω . On the basis of the issues discussed here, I propose that members of SIGACT, and editors of computer science and mathematics journals, adopt the O , Θ , Ω notations as defined above, unless a better alternative can be found reasonably soon.”

— “*Big Omicron and Big Omega and Big Theta*”, SIGACT News, 1976.

Quem Culpar?

Donald Knuth:



- Professor emérito da Universidade de Stanford.
- É o autor do livro *The Art of Computer Programming: American Scientist has included this work among “100 or so Books that shaped a Century of Science”*.
- Prêmio Turing em 1974.
- Análise de Algoritmos e $\text{\LaTeX}$.



greatest computer scientist of all time

[Todas](#)

[Vídeos](#)

[Imagens](#)

[Notícias](#)

[Shopping](#)

[Mais](#)

[Configu](#)

Cientistas da Computação



Alan Turing
1912–1954



Donald Knuth



Tim Berners-
Lee



Ken
Thompson

Relações Assintóticas

Transitividade

Def. (Transitividade): Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$.

Prova.

- É dado que $f(n) \leq c'g(n)$ para todo $n \geq n'_0$ e $g(n) \leq c''h(n)$ para todo $n \geq n''_0$.
- Considere $n_0 = \max(n'_0, n''_0)$, então nós temos que $f(n) \leq c'g(n) \leq c'c''h(n)$.
- Logo, $f(n) \leq c'c''h(n)$ e $f(n) = O(h(n))$. ■

Transitividade

Def. (Transitividade): Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ então $f(n) = \Omega(h(n))$.

P: O que podemos concluir?

Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ então $f(n) = \Theta(h(n))$.

Transitividade

- Se $f(n) = \Theta(g(n))$ e $g(n) = \Theta(h(n))$ então $f(n) = \Theta(h(n))$.
- Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ então $f(n) = \Omega(h(n))$.
- Se $f(n) = O(g(n))$ e $g(n) = O(h(n))$ então $f(n) = O(h(n))$.
- Se $f(n) = o(g(n))$ e $g(n) = o(h(n))$ então $f(n) = o(h(n))$.
- Se $f(n) = \omega(g(n))$ e $g(n) = \omega(h(n))$ então $f(n) = \omega(h(n))$.

Reflexividade

- $f(n) = \Theta(f(n))$.
- $f(n) = \Omega(f(n))$.
- $f(n) = O(f(n))$.
- $f(n) = o(f(n))$? Não.
- $f(n) = \omega(f(n))$? Não.

Simetria e Simetria Transposta

- $f(n) = \Theta(g(n))$ se e somente se $g(n) = \Theta(f(n))$.
- $f(n) = O(g(n))$ se e somente se $g(n) = \Omega(f(n))$.
- $f(n) = o(g(n))$ se e somente se $g(n) = \omega(f(n))$.

Prova ($f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$):

- $f(n) = O(g(n)) \iff \exists c > 0, n_0 \geq 0$ tal que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.
- Dividindo por c : $g(n) \geq \frac{1}{c} \cdot f(n)$ para todo $n \geq n_0$.
- Escolhendo $c' = \frac{1}{c} > 0$, temos por definição que $g(n) = \Omega(f(n))$. ■

Soma de Funções

Def. (Soma de Funções): Se $f(n) = O(h(n))$ e $g(n) = O(h(n))$ então $f(n) + g(n) = O(h(n))$.

Prova.

- É dado que $f(n) \leq c'h(n)$ para todo $n \geq n'_0$ e $g(n) \leq c''h(n)$ para todo $n \geq n''_0$.
- Considere $n_0 = \max(n'_0, n''_0)$, então nós temos que $f(n) + g(n) \leq c'h(n) + c''h(n) = (c' + c'')h(n)$.
- Logo, $f(n) + g(n) = O(h(n))$. ■

Soma de Funções

Def. (Soma de Funções): Seja k uma constante, $f_1(n), f_2(n), \dots, f_k(n)$ e $h(n)$ funções tal que $\forall i, f_i(n) = O(h(n))$. Então $f_1(n) + f_2(n) + \dots + f_k(n) = O(h(n))$.

Consequência:

- Para um algoritmo com duas partes onde é fácil mostrar que uma das partes possui o maior custo.
- Assintoticamente o custo do algoritmo é o custo da parte de maior custo.
- Exemplo: Condicional.

Soma de Funções

Def. (Soma de Funções): Sejam $f(n)$ e $g(n)$ duas funções tal que $g(n) = O(f(n))$. Então $f(n) + g(n) = \Theta(f(n))$.

Prova.

- $f(n) + g(n) = \Omega(f(n))$ já que $f(n) + g(n) \geq f(n)$.
- Já que $g(n) = O(f(n))$ e $f(n) = O(f(n))$, então $f(n) + g(n) = O(f(n))$.
- Logo, $f(n) + g(n) = \Theta(f(n))$. ■

Propriedades Assintóticas de Funções

Polinômios

Def. $f(n) = a_k n^k + \dots + a_1 n^1 + a_0$ para algum $k > 0$ com $a_k > 0$.

Algoritmos Polinomiais:

- $O(n^2)$ e $O(n^3)$.
- $O(n^k)$ onde k não é inteiro.
- $O(n^{1,59})$ e $O(\sqrt{n})$.
- $O(n \log(n))$.

Logaritmos

Def. $\log_b n$ é o número x tal que $b^x = n$.

- $a = b^{\log_b a}$
- $\log_c(ab) = \log_c a + \log_c b$
- $\log_b a^n = n \log_b a$
- $\log_b a = \frac{\log_c a}{\log_c b}$
- $\log_b(1/a) = -\log_b a$
- $\log_b a = \frac{1}{\log_a b}$
- $a^{\log_b c} = c^{\log_b a}$

Logaritmos

Afirmção. Para todo $b > 1$ e todo $k > 0$, nós temos que $\log_b n = o(n^k)$ (portanto $\log_b n \neq \Theta(n^k)$).

Prova.

$$\lim_{n \rightarrow \infty} \frac{\log_b n}{n^k} = 0$$

Intuição:

Logaritmos

Def. $\log_b n$ é o número x tal que $b^x = n$.

Obs. A base do logaritmo não importa.

Afirmção. $\log_a n = \Theta(\log_b n)$.

Prova.

- $\log_a n = \frac{\log_b n}{\log_b a}$
- $\log_a n = \frac{1}{\log_b a} \cdot \log_b n$
- $\frac{1}{\log_b a}$ é uma constante maior que zero que independe de n . ■

Exponenciais

Def. $f(n) = r^n$ com $r > 1$.

Afirmção. Para todo $r > 1$ e todo $k > 0$, nós temos que $n^k = o(r^n)$.

Prova.

$$\lim_{n \rightarrow \infty} \frac{n^k}{r^n} = 0$$

Intuição: O denominador cresce mais rapidamente.

Exponenciais

Def. $f(n) = r^n$ com $r > 1$.

- $r^0 = 1$
- $r^1 = r$
- $r^{-1} = \frac{1}{r}$
- $(r^m)^n = (r^n)^m$
- $(r^m)^n = r^{nm}$
- $r^m \cdot r^n = r^{n+m}$

Exponenciais

Obs. A base da exponencial importa.

Afirmção. Para $r > s$, $r^n \neq \Theta(s^n)$.

Prova. Suponha, para obter contradição, que $r^n = \Theta(s^n)$. Então **existem** constantes $c_1, c_2 > 0$ e n_0 tais que:

$$c_1 s^n \leq r^n \leq c_2 s^n \quad \text{para todo } n \geq n_0.$$

Em particular, teríamos a cota superior $r^n \leq c_2 s^n$, isto é, $\left(\frac{r}{s}\right)^n \leq c_2$ para todo $n \geq n_0$.

Mas como $\frac{r}{s} > 1$, temos $\left(\frac{r}{s}\right)^n \rightarrow \infty$ quando $n \rightarrow \infty$, o que contradiz a existência de tal c_2 independente de n . Logo, $r^n \notin O(s^n)$ e $r^n \neq \Theta(s^n)$. ■

Fatoriais

Obs. Fatoriais crescem mais rapidamente que exponenciais de base fixa.

Afirmção. Para todo $r > 0$, $r^n = o(n!)$ (então $n! \notin \mathcal{O}(r^n)$).

Prova. Para $n \geq 2$:

$$n! = \prod_{k=1}^n k \geq \prod_{k=\lceil n/2 \rceil+1}^n k \geq \left(\frac{n}{2}\right)^{n/2}.$$

Logo, $0 \leq \frac{r^n}{n!} \leq \frac{r^n}{\left(\frac{n}{2}\right)^{n/2}} = \left(\frac{r}{\sqrt{n/2}}\right)^n.$

Como r é constante e $\sqrt{n/2} \rightarrow \infty$, **existe** n_0 tal que $\frac{r}{\sqrt{n/2}} < 1$ para todo $n \geq n_0$, e

então $\lim_{n \rightarrow \infty} \left(\frac{r}{\sqrt{n/2}}\right)^n = 0$. Portanto, $\lim_{n \rightarrow \infty} \frac{r^n}{n!} = 0$, isto é, $r^n = o(n!)$. ■

Principais Classes Assintóticas

Constante - $O(1)$

TROCA(A, x, y)

1: $tmp \leftarrow A[x]$

2: $A[x] \leftarrow A[y]$

3: $A[y] \leftarrow tmp$

Outros exemplos: deletar elemento de um vetor indexado, *pop* de uma pilha,

...

Sublinear - $O(\log n)$

A é um vetor ordenado de tamanho n .

BUSCA-BINARIA(A, n, T)

```
1:   $L \leftarrow 0$ 
2:   $R \leftarrow n - 1$ 
3:  while  $L \leq R$  do
4:     $m \leftarrow L + \lfloor (R - L)/2 \rfloor$ 
5:    if  $A[m] < T$  then
6:       $L \leftarrow m + 1$ 
7:    else if  $A[m] > T$  then
8:       $R \leftarrow m - 1$ 
9:    else
10:
11:      return  $m$ 
12:  return NÃO-ENCONTRADO
```

Por que a Busca Binária é $O(\log n)$?

- A cada passo, o vetor é dividido ao meio.
- Restam $\frac{n}{2}$ elementos, depois $\frac{n}{4}$, depois $\frac{n}{8}$, e assim por diante.
- O processo termina quando sobra apenas 1 elemento.

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \rightarrow \dots \rightarrow 1$$

Quantas divisões? Seja k o número de passos. Temos:

$$\frac{n}{2^k} = 1 \quad \Rightarrow \quad k = \log_2 n.$$

Sublinear - $O(\sqrt{n})$

Grover's algorithm

- **O que é?** Algoritmo quântico de busca não estruturada, criado por Lov Grover em 1996.
- **Ideia principal:** Dado um domínio de tamanho N , encontra (com alta probabilidade) a entrada que leva a um valor de saída desejado usando apenas $O(\sqrt{N})$ avaliações da função.
- **Comparação com computação clássica:** O problema análogo exige $O(N)$ avaliações.

<https://www.youtube.com/watch?v=RQWpF2Gb-gU>

<https://www.youtube.com/watch?v=Dlsa9EBKDGI>

Linear - $O(n)$

```
MERGE(A, start, mid, end)
1:  for i  $\leftarrow$  start to end do
2:    Aux[i]  $\leftarrow$  A[i]
3:    i  $\leftarrow$  start; j  $\leftarrow$  mid + 1; k  $\leftarrow$ 
      start
4:    while i  $\leq$  mid and j  $\leq$  end do
5:      if Aux[i] < Aux[j] then
6:        A[k]  $\leftarrow$  Aux[i]; i  $\leftarrow$  i + 1
7:      else
8:        A[k]  $\leftarrow$  Aux[j]; j  $\leftarrow$  j + 1
9:        k  $\leftarrow$  k + 1
10:   while i  $\leq$  mid do
11:     A[k]  $\leftarrow$  Aux[i]; i  $\leftarrow$  i + 1;
      k  $\leftarrow$  k + 1
12:   while j  $\leq$  end do
13:     A[k]  $\leftarrow$  Aux[j]; j  $\leftarrow$  j + 1;
      k  $\leftarrow$  k + 1
```

- Há um vetor auxiliar *Aux* que recebe a porção do vetor a ser ordenada.
- *i* aponta para o início do primeiro subvetor.
- *j* aponta para o início do segundo subvetor.
- *k* aponta para a posição de saída em *A*.
- O laço principal compara os elementos dos dois subvetores e insere em ordem.

$$O(n \log n)$$

MERGESORT($A, start, end$)

```
1:  if  $start < end$  then  
2:     $mid \leftarrow \lfloor (start + end) / 2 \rfloor$   
    MERGESORT( $A, start, mid$ )  
    MERGESORT( $A, mid + 1, end$ )  
    INTERCALA( $A, start, mid, end$ )
```

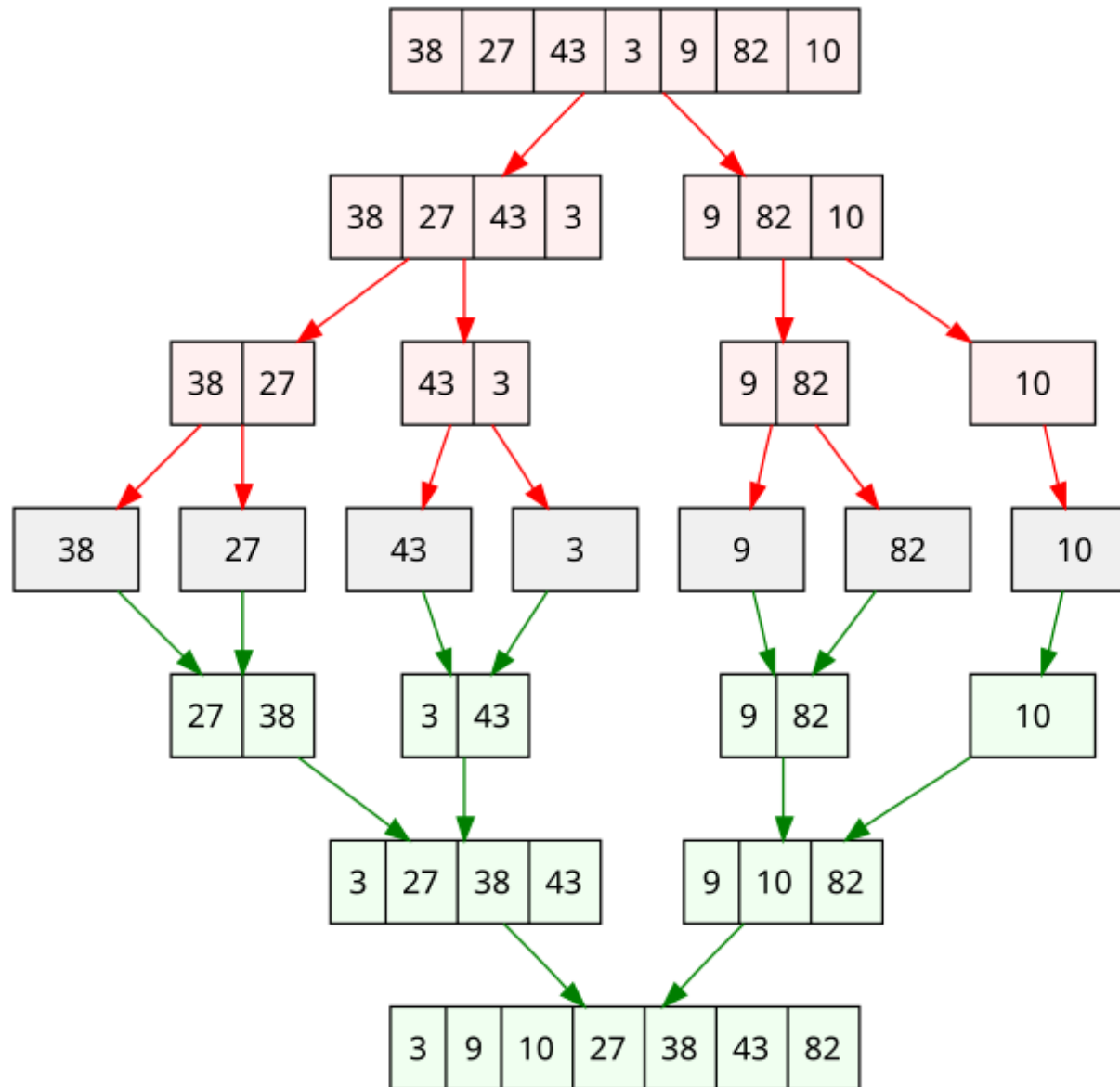
Observações:

- $start$ e end representam os índices de início e fim da parte do vetor a ser ordenada.
- Para ordenar o vetor inteiro: MergeSort($A, 1, \text{length}(A)$).

Complexidade de Tempo do MergeSort (pior caso)

- Em cada nível da recursão, o vetor de tamanho n é dividido em duas partes de tamanho $\approx n/2$.
- O custo de intercalar dois subvetores de tamanho total n é $O(n)$.
- A profundidade da recursão é $\log_2 n$ (pois a cada passo dividimos ao meio).
- Assim, em cada um dos $\log_2 n$ níveis, o custo é $O(n)$.

Complexidade de Tempo do MergeSort (pior caso)



Quadrática - $O(n^2)$

GALE-SHAPLEY(H, E)

```
1:  Inicialize  $S \leftarrow \emptyset$ , e todos os hospitais e estudantes como livres
2:  while existe um hospital livre  $h$  que ainda não propôs a todo estudante do
3:      Escolha um tal hospital  $h$ 
4:      Seja  $e$  o estudante de maior preferência de  $h$  ao qual  $h$  ainda não propôs
5:      if  $e$  está livre then
6:           $S \leftarrow S \cup \{(h, e)\}$ 
7:      else
8:           $e$  está atualmente emparelhado com  $h'$ 
9:          if  $e$  prefere  $h'$  a  $h$  then
10:              $h$  continua livre
11:          else
12:              $S \leftarrow S \setminus \{(h', e)\}$ 
13:              $S \leftarrow S \cup \{(h, e)\}$ 
14:              $h'$  fica livre
15:
16:  return  $S$ 
```

Nota: depende de estruturas de dados adequadas.

Outros exemplos: InsertionSort, BubbleSort, QuickSort*, encontrar o par de pontos mais próximos por força bruta.

Cúbica - $O(n^3)$

Problema: Dados $S_1, S_2, \dots, S_n \subseteq \{1, 2, \dots, n\}$, decidir se **existe** um par (S_i, S_j) disjunto (sem elementos em comum).

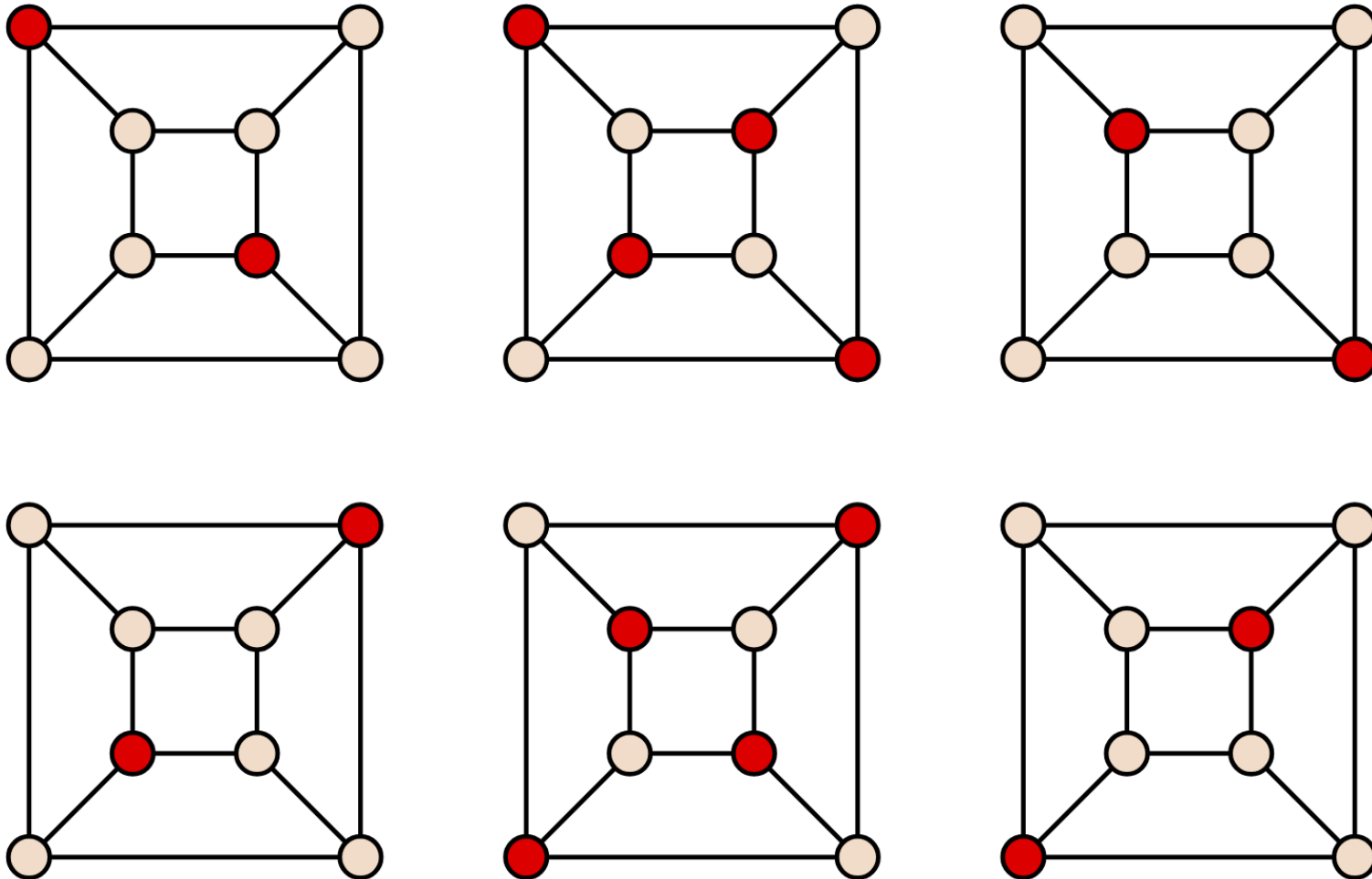
EXISTE-PAR-DISJUNTO($S_1, \dots, S_n$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:    for  $j \leftarrow 1$  to  $n$  do
3:      if  $i \neq j$  then
4:        encontrouComum  $\leftarrow$  false
5:        for cada elemento  $p$  de  $S_i$  do
6:          if  $p \in S_j$  then
7:            encontrouComum  $\leftarrow$  true; break
8:        if  $\neg$ encontrouComum then
9:
10:       return true
11:  return false
```

Nota: **Existem** algoritmos que melhoram o tempo de execução $O(n^3)$ para este problema, mas são bastante complexos.

Exponenciais

Encontrar o conjunto independente máximo de um grafo: $O(2^n)$



Encontrar uma solução exata para o problema do caixeiro-viajante: $O(n!)$

https://en.wikipedia.org/wiki/Travelling_salesman_problem#/media/File:Bruteforce

